

Simulador de Escalonamento de Tempo Real



Luis Sena Antoniel Magalhães



Agenda (10 min)

Bloco	Conteúdo	Quem
Parte 1	Sistemas de tempo real, modelo de tarefas, simulação a eventos e os algoritmos (RM, DM, EDF)	Luis
Parte 2	Arquitetura do simulador, demonstração ao vivo, resultados e validação	Antoniél

Objetivo: simular o escalonamento de tarefas de tempo real por **dois ou mais** algoritmos, avançando **de evento em evento**.

O problema: sistemas de tempo real

- Em tempo real, o resultado certo **tarde** é um resultado **errado**: a correção depende do tempo.
- Modelamos a carga como **tarefas periódicas**. Cada tarefa libera uma sequência infinita de **jobs**.
- Cada job precisa de CPU e tem um **deadline**. A pergunta central:
este conjunto de tarefas é escalonável sem perder nenhum deadline?
- Duas formas de responder: **testes analíticos** (utilização) e **simulação** do escalonamento.

Modelo de tarefas

Cada tarefa τ_i é definida por:

- C_i : tempo de execução no pior caso (*WCET*);
- T_i : período (intervalo entre liberações de jobs);
- D_i : deadline relativo (usamos $D_i = T_i$, deadline implícito);
- ϕ_i : fase (instante da primeira liberação).

Utilização do sistema:

$$U = \sum_{i=1}^n \frac{C_i}{T_i}$$

Fração da CPU exigida pelas tarefas. $U > 1$ é sobrecarga: *impossível* escalonar.

Simulação a eventos discretos

- Em vez de avançar o tempo de 1 em 1 unidade, **saltamos de evento em evento**.
- Entre dois eventos consecutivos, o job de **maior prioridade** executa sem interrupção.
- Eventos que movem a simulação:
 - **Liberação** de job (uma tarefa cria um novo job);
 - **Término** de job (um job termina sua execução).
- Próximo evento = $\min(\text{próxima liberação}, \text{término do job atual})$.
- **Preempção** surge naturalmente: ao liberar um job de maior prioridade, ele assume a CPU.
- Perda de deadline é uma **observação derivada** (não é um evento de escalonamento).

Os algoritmos

- **Rate Monotonic (RM)** – prioridade **estática**: menor período, maior prioridade. (Liu & Layland, 1973)
- **Deadline Monotonic (DM)** – prioridade estática: menor deadline relativo, maior prioridade.
- **Earliest Deadline First (EDF)** – prioridade **dinâmica**: o job com o deadline absoluto mais próximo executa.

Uma só abstração

Toda política é uma **chave de prioridade**: (menor chave = maior prioridade). O motor sempre roda o job de menor chave. RM, DM e EDF diferem *apenas* na chave.

Testes de escalonabilidade

- **RM (suficiente)** – limite de Liu & Layland:

$$U \leq n \left(2^{1/n} - 1 \right) \xrightarrow{n \rightarrow \infty} \ln 2 \approx 0,693$$

Se U está abaixo do limite, RM *garante* escalonabilidade.

- **EDF (exato)** – para deadline implícito:

$$U \leq 1 \iff \text{escalonável}$$

- Por isso EDF alcança **100%** de utilização, enquanto prioridade fixa fica perto de 69% no pior caso.
- A simulação confirma na prática o que o teste prevê na teoria.

Arquitetura do simulador

Python puro (biblioteca padrão), sem dependências. Núcleo de poucas centenas de linhas:

- `model.py` – Task, Job, TaskSet (carrega JSON);
- `schedulers.py` – políticas RM / DM / EDF como chaves de prioridade;
- `simulator.py` – laço de eventos (libera, escolhe, executa, termina);
- `metrics.py` – utilização e testes de escalonabilidade;
- `gantt.py` – diagrama de Gantt em ASCII e SVG;
- `cli.py` – comandos run, compare, analyze.

```
running = min(ready, key=lambda j: policy.key(j, t))  
t_next = min(completion, next_release) # salta para o evento
```

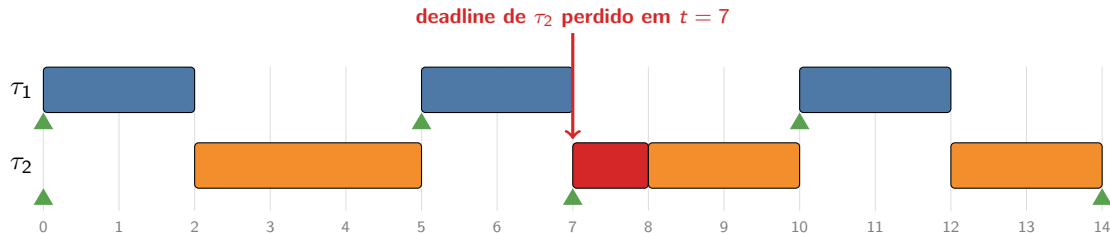
Demonstração ao vivo

Conjunto: $\tau_1 = (C=2, T=5)$ e $\tau_2 = (C=4, T=7)$, com $U = 0,971 < 1$.

```
$ python -m rtsim compare examples/rm_fails_edf_ok.json \  
    --algos RM EDF --verbose
```

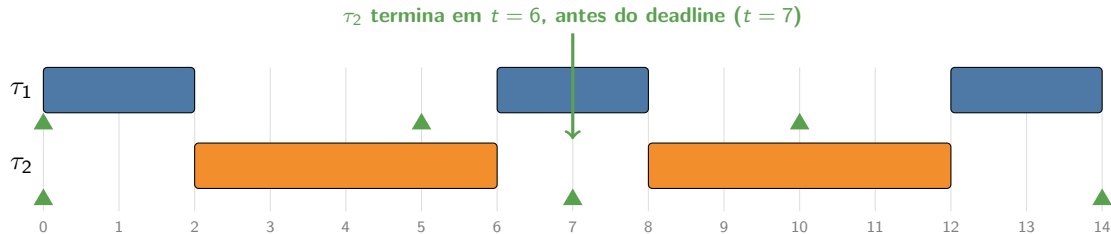
- Mesma carga, dois algoritmos, lado a lado.
- RM (prioridade fixa) e EDF (prioridade dinâmica) tomam decisões diferentes a partir de $t = 5$.
- A seguir, o diagrama de Gantt gerado pelo próprio simulador.

Resultado: RM perde o deadline



Em $t = 5$, τ_1 preempta τ_2 . τ_2 ainda devia 1 unidade quando seu deadline ($t = 7$) chegou: **perda**. Pior tempo de resposta de $\tau_2 = 8 > 7$.

Resultado: EDF cumpre todos os deadlines



EDF prioriza o deadline mais próximo: em $t = 2$, τ_2 (deadline 7) roda antes de τ_1 (deadline 10). Nenhuma perda. Mesma carga, decisão melhor.

Resumo dos resultados

Conjunto	Utilização	RM	EDF
feasible	$U = 0,71$ (abaixo do limite)	cumpre	cumpre
rm_fails_edf_ok	$U = 0,97$ (< 1)	perde	cumpre
overload	$U = 1,10$ (> 1)	perde	perde

- Com $U \leq$ limite, ambos cumprem (como prevê o teste).
- Na faixa $0,69 < U \leq 1$, EDF cumpre onde RM falha.
- Com $U > 1$ (sobrecarga), nenhum algoritmo salva: a teoria e a simulação concordam.

Validação

- **17 testes automatizados** (unittest), todos passando:
 - utilização, limite de Liu & Layland e hiperperíodo;
 - RM escolhe menor período; EDF escolhe menor deadline;
 - conservação: tempo ocupado de CPU = soma dos C executados;
 - segmentos nunca se sobrepõem (um job por vez);
 - o caso âncora: RM perde em $t = 7$, EDF cumpre.
- Verificação cruzada **teoria vs. simulação**: o teste analítico e o resultado simulado coincidem em todos os conjuntos.

```
$ python -m unittest discover -s tests → OK (17 tests)
```

Conclusão

- Simulador a eventos discretos, **de evento em evento**, com liberação e término de jobs.
- **Três** algoritmos (RM, DM, EDF) sob uma única abstração de prioridade.
- Saídas para ensino: trace de eventos, Gantt em ASCII (terminal) e SVG (navegador).
- Demonstra na prática a fronteira clássica: prioridade fixa ($\approx 69\%$) vs. EDF (100%).
- **Trabalhos futuros:** recursos compartilhados (eventos de entrada/saída de recurso) e protocolos de herança de prioridade.



Perguntas

Obrigado!

Luis Sena | Antoniel Magalhães

github.com/luisfelipesena/rt-scheduler-sim